

Adapting Small Vision-Language Models (<1B) for Document Understanding

A systematic evaluation and improvement strategy

Author: Sangbum Choi · Date: 2026-06-21

0. Executive summary

Small VLMs (sub-1B parameters) are attractive for document understanding because documents are high-volume, privacy-sensitive, and increasingly processed on-device or at the edge — exactly where a 3B+ model is too expensive. But their document capabilities are under-characterised: most public leaderboards focus on 3B–70B models.

This report (1) selects ~19 sub-1B-centric VLMs spanning *generalists*, *edge models* and *OCR specialists*; (2) defines a document-understanding benchmark suite anchored on **DocVQA**, **InfoVQA**, **ChartQA** and **OCRBench**, plus **controlled capability/spatial probes** and a **custom robustness probe**; (3) evaluates them with metrics that go beyond accuracy — **ANLS / relaxed accuracy / OCRBench score**, plus **calibration (ECE)**, **robustness retention**, and **shortcut-robust spatial/context signals**; and (4) turns the resulting gap analysis into a concrete, literature-grounded improvement plan that is implemented as a fine-tuning PoC.

Headline finding (measured — full GPU sweep on a free T4). All registered models were run end-to-end (`scripts/run_full_comparison.sh` , captured in [../notebooks/colab_full_comparison.ipynb](https://colab.research.google.com/github/sangbumchoi/notebooks/blob/main/colab_full_comparison.ipynb)). Among **strictly sub-1B** models, **Qwen3.5-0.8B** is the strongest generalist (clears text recognition, KIE, numeric-sum and chart axes), with the **InternVL-1B family (~0.94B)** close behind. The **overall standout is LFM2.5-VL-1.6B** (just over 1B, kept as the upper-bound reference and the Part-2 fine-tuning base): it is the **only model clearing both reasoning axes (numeric-sum H1 and relational-compare H2)**, has the **best grounding** on the proposed custom-eval (spot-loU 0.229 vs ≤ 0.04 for every other model), is **rotation-180 robust** (1.0 retention where most collapse), and is **~14× faster than Qwen3.5-0.8B** (0.98 s vs 13.9 s avg latency on a T4) thanks to its hybrid-conv backbone.

The measured knowledge gap (Part 2 target). Even the strongest models share concrete, falsifiable deficits: **box-tracking (L4) is unsolved by every model**; **grounding (L1) ≈ 0** for all but LFM; **relational reasoning (H2)** fails for most sub-1B models; and **180°-rotation robustness collapses** for most. These — not raw recognition, which is largely solved — are what the Part-2 improvement strategy attacks. **Part 2 selects one of these as its primary objective: spotting / grounding (L1)** — because the extracted fields drive a downstream *action* (DB write, API payload) that needs a **partly-human verification** step, and a reviewer can only check a field efficiently if the model points at the **localised region** the answer came from (§Part 2.1b).

Reproducibility note. Results here are **measured on a free Colab/Kaggle T4** (the resource the task allows), with cached `predictions.jsonl` so re-scoring never re-runs a model and runs are resumable. The evaluation pipeline is complete and tested; the **Part-2**

fine-tuning is preliminary — runs so far are brief (small A0/A1, few images/epochs), enough to read the *direction* of an effect but **not** an effect size (§Part 2.1c), so its findings are stated as weakly-held hypotheses. Where published figures are cited (e.g. authors' DocVQA/OCRBench numbers), they are labelled as such and separated from our measured probe scores in the comparison table.

What is measured in this repo. Beyond the public benchmarks, the report contributes a **capability-axis lens** — a taxonomy of what a document reader must actually do (text, location, hybrid reasoning) instantiated as **controlled probes** with built-in ground truth (§Part 1.2b), plus a **proposed custom-eval** sliced by content-class / language / rotation / reading-direction / spotting. The reproduced signal: **hybrid content-reasoning (sum & compare) is cleared only by LFM2.5-VL-1.6B and MiniCPM-V-4.6, recognition is largely solved across the field, and grounding/box-tracking remain the systemic gap.** See [results_analysis.md](#) / [insights.md](#).

Part 1 — Evaluation

1. Model selection criteria

I select for **architectural diversity within the <1B budget**, so the comparison explains *why* one design wins, not just *which* wins. Three archetypes are represented:

#	Model	Params	Vision encoder + LM	Archetype	Why included
1	InternVL2.5-1B	~0.94B	InternViT-300M + Qwen2.5-0.5B, dynamic tiling	Document generalist	SoTA-for-size doc/OCR; first-class in VLMEvalKit
2	InternVL3-1B	~0.94B	InternViT-300M + Qwen2.5-0.5B, native multimodal pretrain	Document generalist	Newer pretraining recipe; head-to-head with 2.5
3	SmolVLM-500M	0.5B	SigLIP + SmolLM2, Idefics3, heavy token compression	Edge	Edge-deployment anchor; trained incl. Docmatix
4	SmolVLM-256M	0.26B	SigLIP + SmolLM2	Extreme edge	Smallest serious VLM; floor of the size/quality curve
5	LLaVA-OneVision-0.5B	~0.9B	SigLIP-SO400M + Qwen2-0.5B, AnyRes	General VLM	"General-purpose small VLM" baseline (not doc-specialised)
6	GOT-OCR2.0	~0.58B	~80M ViT + Qwen-0.5B decoder	OCR specialist	Pure transcription; tests "OCR ≠ document QA"
7	Florence-2-large	~0.77B	DaViT + BART enc-dec, task tokens	OCR/detection specialist	Unified task-token model; no free-form VQA
8	PaddleOCR-VL-0.9B	~0.9B	NaViT-style encoder + ERNIE-4.5-0.3B	Doc-parsing specialist	Newest, most document-specialised release

Rationale highlights

- **Why InternVL ×2.** It is the consensus strongest small document model, and including both 2.5 and 3 isolates the effect of the *pretraining recipe* at fixed architecture/size — directly relevant to "pretraining data" in the selection criteria.

- **Why two SmolVLM sizes.** They trace the size/quality frontier (256M→500M) and represent the edge-deployment use case the task emphasises; SmolVLM's training mix includes **Docmatix**, a large document instruction set, making it the strongest *small-and-edge* document model and a fair foil to InternVL.
- **Why a general baseline (LLaVA-OV-0.5B).** To quantify the premium document-specialised pretraining buys over a strong but general small VLM at the same ~0.9B size.
- **Why two OCR specialists (GOT, Florence-2) and a parser (PaddleOCR-VL).** Document understanding is *not* OCR. GOT/Florence-2 transcribe but cannot answer questions; including them makes the "**recognition vs. reasoning**" gap measurable rather than asserted, and PaddleOCR-VL tests whether a parsing-first design transfers to QA.

Extended candidate pool (added after review). The harness also registers six further sub-1B open-weight models so the comparison spans the current field, including a *version ablation* of the leading document specialist:

Model	Params	Components	Note
PaddleOCR-VL-1.5	~0.9B	NaViT + ERNIE-4.5-0.3B	v1.5: +polygon localization / text-spotting / seal; OmniDocBench v1.5 ≈ 94.5 (SOTA-tiny)
InternVL2-1B	~0.94B	InternViT-300M + Qwen2-0.5B	older 1B; DocVQA 81.7 / OCRBench 754 — recipe baseline vs 2.5/3
Ovis2-1B	~1.0B	AIMv2-large + Qwen2.5-0.5B	structural visual-text embedding; strong OCR (OCRBench ≈ 89/100)
H2OVL-Mississippi-0.8B	0.8B	InternVL-style + H2O-Danube	OCR/doc specialist (19M pairs); OCRBench 751
SmolDocling-256M	0.26B	SmolVLM-256M base	smallest true <i>document</i> specialist; emits structured DocTags
Florence-2-base	0.23B	DaViT + BART enc-dec	smaller task-token sibling of Florence-2-large

Models documented but **not registered**: *InternVL3.5-1B* (~1.1B, borderline over budget), *Moondream-0.5B* (license unconfirmed via gated card), and the seq2seq specialists *Donut* (~0.2B), *Pix2Struct-base* (~0.3B), *TrOCR* (~0.3-0.6B) — strong but non-conversational, so they need task-specific harnessing rather than the shared VQA loop. **Out of <1B scope** (flagged for completeness): MonkeyOCR-pro-1.2B, Kosmos-2.5 (~1.3B), dots.ocr (~1.7B), Janus-Pro-1B (~1.5B, non-permissive license), Qwen2-VL-2B, Ovis2-2B, InternVL2.5/3.5-2B.

Newer 2025-26 additions (registered for the sweep). Further recent releases were added so the comparison tracks the moving field — including the two models that the measured GPU run made central (the best **strictly sub-1B** generalist and the **Part-2 fine-tuning base**):

Model	Params	Vision encoder + LM	Archetype	Why included
Qwen3.5-0.8B (VL)	~0.87B	Qwen3.5-VL vision tower + 0.8B LM	Generalist (sub-1B)	Best strictly-sub-1B generalist in our sweep (T1/T2/H1/H3 = 1.0); the Part-2 fallback base
LFM2.5-VL-1.6B	~1.6B	SigLIP2 + LFM2 hybrid-conv backbone	Efficient generalist	Part-2 fine-tuning base : only model clearing H1+H2, best grounding (spot-IoU 0.229),

Model	Params	Vision encoder + LM	Archetype	Why included
				rotation-180 robust, ~14× faster than Qwen3.5 on a T4
MiniCPM-V-4.6	~1.3B	SigLIP + MiniCPM LM	Strong reference (>1B)	Co-leader on reasoning (H1+H2) and spatial signals — upper-bound reference
LightOnOCR-1B	~1.16B	Mistral3/ Pixtral-style	OCR specialist	Recent OCR specialist (<code>lightonai/LightOnOCR-1B-1025</code>)

Qwen3.5-0.8B is the only genuinely sub-1B entry of these; MiniCPM/LFM/LightOnOCR sit just over the <1B line and are kept as **stronger reference points / the efficient deployment frontier**. *Ovis2.5-2B* (~2.6B; no 1B variant) has an adapter but is **excluded from the default sweep** (custom interface, well over budget); opt in with `--models ovis2_5-2b`. *Shakti-VLM-1B* was requested but is not publicly available on the Hub, so it could not be registered.

Architecture / pretraining / capability profiles for each model are in **Appendix A**.

2. Benchmark selection & design

Document understanding spans *recognition* (read the pixels), *layout* (where things are), and *reasoning* (combine fields, do arithmetic). No single dataset covers all three, so I use a **suite** plus a **custom probe**:

Benchmark	Source (VLMEvalKit / HF)	Primary metric	What it probes	Why it's here
DocVQA	<code>lmms-lab/DocVQA</code> (DocVQA, val)	ANLS	Printed text + layout + KIE on scanned business docs	The canonical document-QA benchmark
InfoVQA	<code>lmms-lab/DocVQA</code> (InfographicVQA, val)	ANLS	Dense multi-element layout, text+chart fusion, numeric reasoning	Hardest layout/reasoning; complements DocVQA
ChartQA	<code>lmms-lab/ChartQA</code> (test; human/aug slices)	Relaxed acc	Reading + arithmetic over plotted data	Structured-visual reasoning
OCRBench	<code>echo840/OCRBench</code>	OCRBench (/ 1000)	Regular/irregular/ handwritten/artistic text, KIE, handwritten math	Fine-grained <i>recognition</i> capability
Robustness probe	derived from DocVQA (ours)	ANLS retention	Capture-quality + terminology stability	The "beyond accuracy" deployment axis

Why validation splits. DocVQA/InfoVQA **test** labels live behind an eval server; I use **val** so results are self-contained and reproducible. (Published model cards often quote *test* — a split mismatch I flag explicitly in the comparison table so numbers are not silently conflated.)

The custom robustness probe (design decisions). Real documents are phone photos, faxes, and re-compressed PDFs, and users phrase questions in domain jargon. Headline ANLS

on pristine scans hides this. The probe takes N DocVQA items and emits a **paired** set — a `clean` baseline plus controlled perturbations — so failures are *attributable*:

- **downscale** (low-DPI / small photo → small-text legibility)
- **jpeg** quality 18 (heavy re-compression artifacts)
- **blur** (out-of-focus capture)
- **rotate** 5° (skew / handheld tilt)
- **noise** (sensor / photocopier speckle)
- **term_paraphrase** (image untouched; question rewritten with terse domain wording — "total"→"aggregate", "how much"→"what is the amount" — a deterministic, rule-based terminology stress test)

Robustness is reported as **retention** = **score(perturbed)** / **score(clean)** per family, which separates "accurate" from "accurate *and stable*".

2b. Capability-axis probes — isolating *what* a reader must do

Public benchmarks bundle many skills into one ANLS number, so a low score does not say *which* ability failed. To make the gap analysis falsifiable on **owned, GT-exact data**, the suite adds a capability-axis catalogue and two controlled probes (full design in [capability_axes.md](#)). The axes are grouped into three families with sequential codes:

Family	Axes	What it isolates
T · Text	T1 text-recognition · T2 KIE-localized	read an exact string / one field's value
L · Location & space	L1 grounding · L2 absolute-region · L3 relative-position · L4 box-tracking	where things are, and spatial relations
H · Reasoning & context	H1 content-sum · H2 content-compare · H3 chart-value · H4 consistency · H5 anti-hallucination · H6 disambiguation · H7 cross-reference	reasoning over read values + context control

- **capability_probe** — measured-score axes (T1/T2, H1/H2/H3, L1) on a synthetic invoice + chart whose pixel boxes are known exactly. Content reasoning (H1/H2) is kept **strictly independent** of position/context: it is arithmetic/comparison over read *values*, not layout.
- **spatial_context_probe** — the signal axes (L2-L4 / H4-H7), each paired with a **shortcut control** (counterfactual / distractor / position-bias) so a pass means the model *cleared the control*, not that it guessed from a language prior. For grounding (L1) every model gets the same normalised "return [x1,y1,x2,y2]" instruction, with native-spotting outputs mapped back to pixels — a fair comparison across chat VLMs and spotting-capable specialists.

Because both probes are rendered here (HTML/CSS → digital-native PDF → exact boxes), they double as **fine-tuning supervision** in Part 2 — the same generator produces evaluation *and* training GT (§Part 2.3).

3. Evaluation metrics — beyond accuracy

Metric	Definition	Why it matters for documents	Status in this PoC
ANLS			used (KIE / short-answer axes)

Metric	Definition	Why it matters for documents	Status in this PoC
	1 – normalised edit distance to best gold; 0 below 0.5	Official DocVQA/InfoVQA metric; tolerant of minor OCR/format noise (e.g. \$1,200 vs 1200) without rewarding near-misses	
NED	1 – normalised edit distance (no 0.5 floor)	Character-level closeness for OCR/transcription where ANLS's hard floor is too harsh	used (T1 / handwriting / multilingual)
Relaxed accuracy	Numeric within 5% rel. error, else exact match	Official ChartQA metric; the right notion of "correct" for read-off numbers	used (H1 / H3)
Exact / TEDS / grounding-IoU	exact string / table tree-edit sim / box IoU	strict answers, table structure, and spotting localisation respectively	used (counts, tables, L1)
OCRBench score	Gold string $\subseteq$ prediction, summed / 1000	Standard OCRBench scoring; isolates <i>recognition</i> from <i>reasoning</i>	not exercised — applies only to the OCRBench dataset, which we cite (published) but did not reproduce ; no probe uses this scorer
Calibration — ECE	weighted mean over 10 confidence bins of abs(accuracy – confidence)	A deployable reader must <i>know when it is unsure</i> so low-confidence fields route to human review; equal-accuracy models can differ sharply in ECE (Guo et al., ICML'17)	partially used — computed per-model in Part-1 <code>summary.json</code> for logit-exposing backends (n/a for OCR specialists); not surfaced in the headline tables, not logged in fine-tuning
Robustness retention	score(perturbed) / score(clean), per perturbation family	Production inputs are degraded; retention predicts real-world accuracy better than clean ANLS	partially used — the rotation-retention slice runs on the custom-eval; the full degraded-input robustness sweep is wired but not in the headline run
Operational	answer-rate, load time, avg latency/sample	Edge viability: a model that's accurate but 10x slower may be unusable	used (efficiency table)

Honesty note on metric coverage. The headline of this PoC is the controlled **probe / custom-eval** family, whose samples are scored with **ANLS / NED / exact / relaxed-acc / TEDS / grounding-IoU**. Two metrics are listed for completeness but are **not part of the measured probe results**: **OCRBench score** is only meaningful on the OCRBench dataset (we report its *published* figures in the comparison table and did not re-run that sweep), and **ECE** is genuinely computed in the Part-1 capability/probe runs (`docs/results/<model>/<probe>/summary.json` — e.g. SmoIVLM-256M ≈ 0.43) but is **not pulled into the headline matrices and is not part of the fine-tuning W&B stream**.

Confidence for ECE is the **mean token probability** of the generated answer, read from HF `generate(output_scores=True)` . Genuinely autoregressive backends (InternVL, SmoIVLM, LLaVA-

OV, PaddleOCR-VL) expose it; transcription-only models (GOT, Florence-2) do not, and are reported as `ECE = n/a` rather than with a fabricated number.

4. Experimental setup & reproducibility

- **Hardware.** Free **Google Colab / Kaggle T4 (16 GB)** — every model fits in fp16/bf16; no paid hardware required, as the task allows.
- **Decoding (held constant across all models for fairness).** Greedy (`do_sample=False`, `num_beams=1`), `max_new_tokens=256`, bf16. Greedy $\Rightarrow$ deterministic $\Rightarrow$ reproducible.
- **Preprocessing.** Each model uses **its own native, documented** image pipeline (InternVL dynamic tiling up to 12×448px; SmolVLM/LLaVA AnyRes; Florence/GOT task pipelines). Forcing a shared resize would unfairly cripple tiling-based models whose document edge *comes from* high-res tiling — so "consistency" here means *consistent decoding + consistent scoring*, not identical pixels. This choice is documented so it is auditable.
- **Determinism.** Fixed seeds; greedy decoding; pinned package versions (`requirements.txt`); predictions cached to `predictions.jsonl` so re-scoring never re-runs the model and runs are resumable.
- **Fairness controls.** Same prompt template per task; same sample subset (`--limit`) across models; identical metric code for all.

5. Software stack

Tool	Role	Why
PyTorch + HF Transformers	Model loading/ inference	Every candidate ships HF weights; one API (<code>AutoModel*</code> , <code>generate</code>) covers them via thin adapters
HF Datasets	Benchmark sourcing	Canonical hosting for DocVQA/InfoVQA/ChartQA/OCRBench
VLMEvalKit (reference only — not a dependency)	Convention reference	We do not run or wrap VLMEvalKit ; we borrow its conventions — which datasets (DocVQA/InfoVQA/ChartQA/OCRBench), the ANLS/relaxed-acc metric definitions, and the concise-answer prompting — and built a standalone <code>docvlm_eval</code> pipeline instead, because the metrics we need (calibration ECE, paired robustness retention, our capability/spatial probes) are not in VLMEvalKit
PEFT (LoRA)	Part-2 fine-tuning PoC	Parameter-efficient adaptation fits the sub-1B/edge story and a T4 budget
Pillow / NumPy / torchvision	Image I/O + perturbations + tiling transforms	Lightweight, ubiquitous

Why a custom pipeline, not VLMEvalKit. VLMEvalKit is the field's standard harness, and we deliberately **align with its conventions** (the same datasets, the ANLS/relaxed-acc definitions, concise-answer prompting) so results are comparable in spirit. But we **do not use it as the harness**: the axes this task actually cares about — **calibration (ECE)**, the **paired robustness retention** probe, and our **capability / spatial-context / spotting probes with built-in GT** — are not in VLMEvalKit's metric set, and bolting them on would be more work than a focused tool. So the deliverable is the small, tested `docvlm_eval` package, whose single `evaluate.py` "loads any model, runs the benchmark, outputs per-model scores" exactly as the PoC requires — VLMEvalKit stays a *reference for choices*, not a runtime dependency.

6. What actually ran (measured, GPU)

The full comparison was run on a **free T4** (`scripts/run_full_comparison.sh` , captured in [../../notebooks/colab_full_comparison.ipynb](https://colab.research.google.com/notebooks/colab_full_comparison.ipynb)): **19/19 models** produce committed results across the capability probe, the spatial/context probe, and the proposed custom-eval, scored with the T/L/H taxonomy. Full read-out: [results_analysis.md](#) · [insights.md](#) · [../results/matrix_capability.md](#) · [../results/probe_signals.md](#) · [../results/custom_eval_breakdown.md](#).

- **Reasoning (capability probe, 1 sample/axis — directional):** numeric-sum **H1** is cleared by many (Qwen3.5, LFM, MiniCPM, InternVL2/2.5, SmolVLM-500M), but relational-compare **H2** is cleared by **only LFM2.5-VL-1.6B, MiniCPM-V-4.6 and SmolVLM-500M** — and *both* H1+H2 only by **LFM and MiniCPM**. (Note: the earlier CPU re-score had credited InternVL2.5/3 with both; the measured GPU run does not — a correction this revision applies.) Recognition (T1/T2) and clean chart-read (H3) are largely solved across the field.
- **Grounding / space: L1 grounding ≈ 0** for every model on the capability probe; on the proposed custom-eval only **LFM reaches a usable spot-ioU (0.229)** vs ≤ 0.04 for all others. On the spatial/context probe, **box-tracking L4 is unsolved by every model**; the strongest shortcut-robust profiles are **LFM and MiniCPM** (clear L2/L3/H5/H6/H7), then InternVL2.5-1B; the best **sub-1B** is **Qwen3.5-0.8B** (L2/H5/H6).
- **Efficiency frontier:** LFM2.5-VL-1.6B (0.98 s/sample) and the Florence/H2OVL specialists are the only fast options; the InternVL-1B family is 6–8 s, Qwen3.5-0.8B ~ 13.9 s, and the PaddleOCR-VL parsers 80–115 s (full-page parsing, not short-answer). This latency spread — and *why* the smaller Qwen is slower than the larger LFM — is dissected in [../../notebooks/latency_profile.ipynb](https://colab.research.google.com/notebooks/latency_profile.ipynb).

This measured picture confirms the gap analysis below: the small models' deficit is **grounding/box-tracking and relational reasoning over recognised content**, not recognition.

Part 2 — Knowledge gap & improvement strategy

1. Knowledge-gap analysis (evidence-based)

Grounded in the **measured** GPU sweep (capability/spatial probes + custom-eval) and corroborated by published figures, four gaps emerge — and they are *capability* gaps, not recognition gaps.

Gap A — grounding & box-tracking (the primary, systemic gap). **L1 grounding ≈ 0** on the capability probe for every model; on the proposed custom-eval only LFM reaches a usable **spot-ioU 0.229** (all others ≤ 0.04). **L4 box-tracking is unsolved by every model**. General small VLMs have no usable spotting head, yet field-localisation/parsing is central to document understanding — this is the highest-leverage gap.

Gap B — relational reasoning over read values (H2). Numeric-sum (H1) is widely solved, but relational-compare (**H2**, "which is largest?") is cleared by only **LFM and MiniCPM**; most sub-1B models answer the *first* item. The bottleneck is multi-region comparison, not OCR — and it tracks the well-known **InfoVQA $\ll$ DocVQA** layout-reasoning deficit (InternVL2.5-1B 56.0 vs 84.8 published).

Gap C — orientation / robustness. 180°-rotation retention collapses for most models (InternVL ≈ 0.06 – 0.10 , SmolVLM ≈ 0.1 – 0.13) while LFM/MiniCPM/PaddleOCR hold 1.0. Combined with the degraded-input robustness probe, this is the deployment-stability axis leaderboards ignore.

Gap D — reliability is under-measured (and, here, only partially measured). No public card reports **calibration (ECE)** or per-perturbation **retention**, yet both decide deployability. Our pipeline *can* measure both, but to be honest about what this PoC actually produced: **ECE is computed** per-model in the Part-1 probe runs (`docs/results/<model>/<probe>/summary.json`, real values for logit-exposing backends; n/a for OCR specialists) **but is not yet surfaced in the headline tables**; and of **retention**, only the **rotation slice** is in the headline custom-eval — the full degraded-input sweep (downscale/jpeg/blur/noise/terminology-paraphrase) is **wired but not yet run** (§3 "Status in this PoC"). So Gap D is **identified and partially instrumented, not fully quantified** — surfacing ECE and running the perturbation sweep is itself a near-term to-do. Separately, recognition $\neq$ reasoning is confirmed by the OCR specialists (GOT/Florence-2/PaddleOCR-VL), which transcribe well but cannot answer questions — pushing OCR alone will not close these gaps.

Which model do we improve, and why LFM is the fine-tuning base. Per the task, Part 1 ranks **strictly sub-1B** models and the best is **Qwen3.5-0.8B** (with InternVL3-1B close). The improvement *methodology* (synthetic GT-exact supervision + LoRA placement, §2) is **model-agnostic** — the A5 placement resolver buckets any model's modules by introspection — so it transfers to the sub-1B winner. We **demonstrate it on LFM2.5-VL-1.6B** because: (i) on a free T4 it is the *only* base that fine-tunes at a feasible rate — Qwen3.5-VL's full-attention prefill runs ~ 0.05 it/s (hours/epoch) vs LFM's hybrid-conv ~ 10 – $14\times$ faster (see [../notebooks/latency_profile.ipynb](https://github.com/lmstudio-org/notebooks/blob/main/latency_profile.ipynb)); and (ii) LFM is already the strongest base on exactly the gap axes (only model with non-trivial grounding + both reasoning axes + rotation robustness), so adapting it is the highest-leverage, lowest-regression target. The same arms run on Qwen3.5-0.8B with `--models qwen3_5-0.8b`.

1b. The selected metric — spotting / grounding (and why)

Of the gaps above we **select grounding / spotting (L1, scored by box IoU)** as the primary metric to improve and report. The reason is the *downstream use*, not just that the number is low:

- Document understanding rarely ends at "read the value" — the extracted fields drive an **action**: writing a row to a **database**, populating an **API header/payload**, triggering a workflow. Any such write needs a **consistency / validation step**, and in practice that step is *partly human*: a reviewer signs off on the fields before they are committed.
- When a human verifies a field, the answer almost never spans the whole page — it lives in a **very localised region** (one cell, one stamp, one line). Making the reviewer hunt the whole document is the bottleneck. What helps is a **"where" clue**: the model pointing at the exact region its answer came from — i.e. a **spotting box** (and, secondarily, a short **rationale** naming the region).
- So spotting is not a cosmetic add-on: it is the signal that makes the model's output **auditable**, turning "trust the answer" into "verify the answer here". That is why we improve and measure L1 grounding (and pair it with the A2 reasoning-clue), rather than chasing already-near-saturated OCR.

This makes **A1 spotting** the lead ablation, with **A2 reasoning** as the supporting "which region / why" clue; the other arms (A4/A5/A7) remain available but secondary to this objective.

1c. Preliminary fine-tuning observations (W&B — brief, not yet conclusive)

Caveat (read first). The fine-tuning experiments run so far are **brief and small** — short LoRA runs on a single T4, few images and few epochs. The numbers below are **preliminary**; they are enough to sanity-check direction, **not** to claim an effect size, and one anomaly (held-out > train on grounding for the vision arm) is almost certainly small-sample noise. Re-run at larger scale before any firm conclusion. (Live curves: [W&B project docvbm-ablation](#); metric meanings in [wandb_metrics.md](#).)

What was run. An **A5 placement** comparison on the LFM spotting arm: the same grounding-aware LoRA training, applied to the **vision encoder** vs the **connector/projector**, scored per-axis on the realistic train and held-out splits. This directly asks *which module to adapt to inject spotting*.

Signal (W&B; held-out unless noted)	vision LoRA	connector LoRA	Reading (hedged)
grounding spot-IoU (held-out)	0.140	0.079	vision ~1.8× connector — grounding <i>appears</i> to favour adapting the vision encoder
grounding spot-IoU (train)	0.120	0.024	connector barely fits boxes even on train → weak grounding pathway
L1-locate (tight box)	train 0.163 / eval 0.023	train 0.0005	vision learns <i>exact</i> localisation; connector ≈ 0
L1-region (enclosing box)	train 0.061	train 0.002 / eval 0.058	both low; the looser region target scores a little higher, as expected
kie ("what", transfer, held-out)	0.986	0.986	recognition held under both placements — "where" supervision didn't cost "what"
overall score (held-out)	0.428	0.380	vision higher overall, driven mostly by the grounding axis

Working hypotheses (weakly held, to be tested at scale):

- *H1 (tentative).* Spotting seems **teachable with a small LoRA** — grounding is non-zero and the vision arm roughly doubles the connector arm on held-out spot-IoU — i.e. it looks like a trainable pathway, not an architectural wall.
- *H2 (tentative).* The grounding pathway *appears* to run through the **vision encoder** more than the connector (vision > connector on every grounding row, starkly on L1-locate). This is consistent with the A5 prior "spatial/grounding ← vision (+connector)", but two arms at small scale can't yet rule out the connector contributing when combined.
- *H3 (tentative).* The lift *appears cheap to recognition*: kie is identical (0.986) across placements, so adding box targets did not visibly erode "what". A real cost may only show at scale.
- *Caveat on magnitude.* Absolute spot-IoU is still low ($\leq \sim 0.14$ held-out) and the train/held-out ordering is noisy, so treat these as *direction*, not achieved performance. The next step is more images/epochs and a vision + connector arm to test H2 directly.

These are framed as conjectures on purpose: with the current brief runs we can observe *direction*, not magnitude. We flag this so the report does not overclaim.

A staged plan, each step tied to a gap and to literature, all runnable on a single T4 — and backed by the **model-agnostic LoRA fine-tuning subpackage already in this repo** (`src/docvlm_eval/finetune`, driven by `scripts/run_ablation.py`).

Two complementary data sources feed the plan, each with a dedicated notebook: - **Synthetic, GT-exact** (`scripts/make_realistic_cases.py`): HTML/CSS → digital-native PDF → exact pixel boxes → Augraphy degradation. Because the generator *authors* every value, it is the **only source that carries spotting boxes (A1) and chain-of-thought rationales (A2) by construction** — and a model-free reasoning engine (`docvlm_eval.synth.reasoning`) emits varied count/aggregate/compare questions per document. Ablations run in [../notebooks/finetune_ablation.ipynb](#). - **Real public benchmarks** (`scripts/build_benchmark_trainset.py`): a small subset (<200 images/benchmark) of every catalog dataset normalised into our training DTO, used to **train-on-public / validate-on-synthetic** in [../notebooks/finetune_ablation\(public_dataset\).ipynb](#). Public data is **feasibility-gated**: it has no boxes/rationale, so it can run A0/A5/A7 but not A1/A2 — exactly the division of labour the two notebooks make explicit.

Step 1 — Targeted LoRA SFT on reasoning/grounding data (attacks Gaps A & B). Parameter-efficient **LoRA** (Hu et al., 2021) / **QLoRA** (Dettmers et al., 2023), placement resolved per-model by introspection (vision / connector / LM-attn / LM-MLP). Train on the reasoning-and-spotting-rich synthetic mix (and the public benchmark subset for real-distribution coverage). Rationale: recognition is already strong; LoRA cheaply re-weights the *reasoning* and *grounding* pathways without catastrophic forgetting, and fits a T4. *Keep high-res handling on* — small-text legibility is what the A7 preprocessing arm controls.

Step 2 — Reasoning distillation from a larger teacher (amplifies Gap A). Generate **chain-of-thought rationales** for InfoVQA/ChartQA train questions with a larger teacher (InternVL-8B/26B or a frontier VLM) and fine-tune the 1B student on (image, question → rationale → answer). Sequence-level KD / rationale distillation transfers *reasoning procedure*, not just answers, and is well-suited to small students (Hinton et al., 2015; Hsieh et al., "Distilling step-by-step", 2023). This directly addresses the multi-step numeric reasoning that InfoVQA exposes.

Step 3 — Robustness-aware augmentation (attacks Gap C / retention). Augment SFT with the **same perturbations as the robustness probe** (downscale/jpeg/blur/ rotate/noise) and **terminology paraphrases**. Standard augmentation theory + the paired probe give a direct read on whether retention improves. Keep augmented and clean copies so clean accuracy is preserved.

Step 4 — Calibration (attacks Gap C / ECE). Post-hoc **temperature scaling** on a held-out doc set (Guo et al., 2017) — one parameter, no accuracy change, large ECE reduction — optionally with **confidence-aware decoding** so the deployed reader can abstain/route low-confidence fields. Re-measure ECE with the pipeline.

Why this combination. LoRA+QLoRA = T4-feasible and forgetting-safe; distillation injects the reasoning the small LM lacks; augmentation + temperature scaling convert clean-set wins into *deployable* wins. The plan is *surgical* — it spends capacity on the one axis the evidence says is weak (InfoVQA/layout reasoning), not on already-saturated OCR.

2b. From strategy to controlled ablations (one factor at a time)

The steps above are not run as one big change — each is an **isolated ablation** with a held-out control, then the winners are stacked into a cumulative **staircase** (full registry and dependency graph in [ablation_plan.md](#), `configs/ablations.yaml`):

Step / question	Ablation	Factor varied	Data switch
Memorization vs understanding (prerequisite)	A0	training-data scale (curve); train-set vs held-out gap	data size
Spotting supervision (where)	A1	target value + [x1,y1,x2,y2] vs value	emit_spotting
Reasoning distillation (Step 2)	A2	target rationale → answer vs answer	emit_rationale
Are the signals complementary?	A3	the four corners of {spot} × {reason}	both flags
Multilingual mix & transfer	A4	{en}, {en+es}, {ko+en}, ... equal totals	languages
LoRA placement (Step 1)	A5	vision / connector / LLM-attn / LLM-MLP	(training)
Hyperparameters (Step 1)	A6	rank/alpha/lr/epochs	(training)
Preprocessing/resize (keep tiling)	A7	dynamic tiling vs fixed; resolution; aspect	render knobs

Data & experiment machinery. Each arm is a synthetic dataset variant whose ground truth *carries the factor being varied*, generated by the same digital-native pipeline that backs the probes — so labels (incl. exact boxes) are free and a degraded/resized copy keeps valid GT. The factors are stored as a typed **DocSample DTO** and controlled by a single config, so an arm differs from its control in exactly one factor family (design: [synthetic_data_dto.md](#)):

```
python scripts/make_realistic_cases.py --config configs/synth_data.yaml --ablation A1_spotting_on
python scripts/make_realistic_cases.py --config configs/synth_data.yaml --ablation A2_reasoning_on
# ...A3_*, A4_ko_en, A7_dynamic_tiling – base + ablation_overrides in one file
```

Every variant is scored by the **same** Part-1 pipeline (capability probe, spatial/context signals, public-benchmark suite), so the staircase is apples-to-apples and each step's height is that factor's marginal contribution. Steps 3–4 (robustness augmentation, temperature-scaling calibration) are layered on top of the chosen training arm and re-measured the same way.

3. Expected outcomes & measurement

Axis	Baseline (measured, T4)	Target after plan	Measured by
L1 grounding / spot-IoU	LFM 0.229; ≤0.04 all others; probe ≈ 0	materially > baseline via A1	capability probe / custom-eval IoU
L4 box-tracking (probe)	0.00 every model	first non-zero via A1	spatial probe
H2 content-compare (probe)	LFM/MiniCPM pass; most sub-1B fail	lift the base via A2	capability probe
180°-rotation retention	LFM/MiniCPM 1.0; InternVL ≈ 0.06–0.10	lift the base via A7	custom-eval rotation
InfoVQA (val ANLS, published)	~56 (InternVL2.5-1B)	+6–10 ANLS	evaluate.py on InfoVQA

Axis	Baseline (measured, T4)	Target after plan	Measured by
DocVQA (val ANLS)	~85 (InternVL2.5-1B)	no regression (± 1)	evaluate.py on DocVQA
Robustness worst-case retention	TBD (pipeline)	+0.1-0.2	robustness probe retention
Calibration (ECE)	TBD (pipeline)	halved	ECE in summary.json

The success criterion is **held-out spotting (L1 / spot-loU) materially above the base**, with recognition (T1 / kie) held and the A2 reasoning-clue improving region attribution — and, as a *prerequisite* (A0), evidence that the gain reflects *localising* rather than memorising box coordinates. Everything is verified by re-running the *same* pipeline, so before/after is apples-to-apples. Fine-tuning numbers are **preliminary** (see §1c) until larger runs land.

Deprecated: the cumulative "staircase". Earlier drafts proposed a headline figure stacking A1–A7 winners into a monotone staircase on a composite metric. With the report **refocused on the single selected objective — spotting for human-in-the-loop verification** — that multi-arm composite is **no longer the headline** and is retired (the plot_ablation.py staircase remains in the repo for optional later use, but is not part of this report's claim). The deliverable is now the **before/after on held-out spot-loU** (§1c table), not a cumulative curve.

Appendix A — Model profiles

- **InternVL2.5-1B / InternVL3-1B** (OpenGVLab/InternVL2_5-1B, OpenGVLab/InternVL3-1B, MIT). InternViT-300M vision encoder + Qwen2.5-0.5B LM via an MLP pixel-shuffle projector, with **dynamic high-resolution tiling** (up to 12× 448px crops + thumbnail). Trained on large multimodal corpora with heavy OCR/document/chart data; v3 adds a native multimodal pretraining recipe. Strong OCR/doc/chart for the size.
- **SmolVLM-256M / 500M** (HuggingFaceTB/SmolVLM-*-Instruct, Apache-2.0). SigLIP vision encoder + SmolLM2 LM in the Lefics3 framework, with aggressive pixel-shuffle token compression for on-device use; training mix includes **Docmatix** (document instructions).
- **LLaVA-OneVision-0.5B** (llava-hf/llava-onevision-qwen2-0.5b-ov-hf, Apache-2.0). SigLIP-SO400M + Qwen2-0.5B with AnyRes tiling; strong general single-image/multi-image/video VLM, not document-specialised.
- **GOT-OCR2.0** (stepfun-ai/GOT-OCR-2.0-hf, Apache-2.0). ~80M ViT encoder + Qwen-0.5B decoder trained end-to-end for **transcription** (plain/formatted/markdown). No free-form question interface → OCR-specialist baseline.
- **Florence-2-large** (microsoft/Florence-2-large, MIT). DaViT encoder + BART-style enc-dec trained on FLD-5B with a **task-token** interface (<OCR>, <OD>, ...). No <DocVQA> token → driven with <OCR>; specialist, not conversational.
- **PaddleOCR-VL-0.9B** (PaddlePaddle/PaddleOCR-VL). NaViT-style dynamic-resolution encoder
- ERNIE-4.5-0.3B LM, purpose-built for full-page **document parsing** (text/table/formula/chart, reading order) across 100+ languages; evaluated by its authors on parsing (OmniDocBench edit-distance), not VQA.

Appendix B — Selected references

- Mathew et al. *DocVQA: A Dataset for VQA on Document Images*. WACV 2021.
- Mathew et al. *InfographicVQA*. WACV 2022.
- Masry et al. *ChartQA*. ACL Findings 2022.
- Liu et al. *OCRBench: On the Hidden Mystery of OCR in Large Multimodal Models*. 2023.
- Chen et al. *InternVL / InternVL 2.5* (arXiv:2412.05271) / *InternVL3* (arXiv:2504.10479).
- Marafioti et al. *Smo/VLM*. arXiv:2504.05299.
- Li et al. *LLaVA-OneVision*. arXiv:2408.03326.
- Wei et al. *General OCR Theory (GOT-OCR2.0)*. arXiv:2409.01704.
- Xiao et al. *Florence-2*. arXiv:2311.06242 (CVPR 2024).
- *PaddleOCR-VL Technical Report*. arXiv:2510.14528.
- Hu et al. *LoRA*. arXiv:2106.09685. Dettmers et al. *QLoRA*. arXiv:2305.14314.
- Hinton et al. *Distilling the Knowledge in a Neural Network*. 2015. Hsieh et al. *Distilling Step-by-Step*. ACL 2023.
- Guo et al. *On Calibration of Modern Neural Networks*. ICML 2017.

Published scores quoted in this report are self-reported by model authors (papers/cards), evaluated via VLMEvalKit/OpenCompass; split conventions (val vs test) and scale conventions (OCRBench /1000 vs /100) differ between sources and are flagged in the comparison table.